Computer Programming 1

References and Pointers

Outline

- References
- Pointers

Data types

- Data types
  - Fundamental data types
  - Derived data types (more complex types can be derived from the fundamental types)
- Constructs that can be used to build derived types are:
  - References
  - Pointers
  - Arrays
  - Structures
  - Unions
  - Classes
- Constructs (listed below) that are used to build user-derived data type
  - Structures
  - Union
  - Classes

Data types

- Data types
  - Fundamental data types
  - Derived data types (more complex types can be derived from the fundamental types)
- Constructs that can be used to build derived types are:
  - **References**
  - **Pointers**
  - Arrays
  - Structures
  - Unions
  - Classes
- Constructs (listed below) that are used to build user-derived data type
  - Structures
  - Union
  - Classes

The type *Reference to*

- The reference mechanism allows you to give multiple names to a variable (or to an expression with an address)
  - A reference is a synonym for the expression to which it refers

    → by modifying one, the other also is modified ("aliasing")
  - A reference is an expression with an address
- Syntax: type & id = exp;
  - exp is an expression with an address
  - type is a type of the language



Example

```
int x=1;
int &y=x;        //y is of type reference-to x, i.e., it is a synonym of x
y = 6            // x is also modified
```

Constraints on the use of *References*

- In the declaration, it is mandatory the initialization
  - E.g., int &y; //wrong!
  - E.g.,
    int x=1;
    int &y =x; //ok
- It is not possible to redefined an already defined variable of type reference
  - double x1,x2;
  - double &y=x1; //ok
  - double &y=x2; //wrong! It has been already defined
- In two cases, it is not (more) possible to defined a reference-to:
  - An expression without an address
    - float &y=10.2; // wrong
  - An expression with an address but of different type
    - double d=3.1; // ok
    - int &z=d; // wrong

Example of use of references:

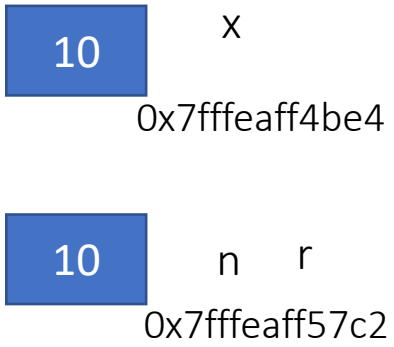
{ ../RIF_POINT/reference.cc}

The operator *address-of «&» (1)*

- The operator “&” (address-of) returns the address (I-value) of the expression where it is applied
- It can be applied only to expression with an address
- It is different with respect to the use of “&” in the definition of references
- Typically, an address requires 4 bytes

Example

```
int x=10;
cout << &x << endl;              // it prints the address of x (not its content)
//cout << &(x*5) << endl;        // wrong! Compile-time error.
int n = 10;
int& r = n;                      // r is the alias of n, it ‘‘points’’ to the same memory area
cout << "&n = " << &n << ", &r = " << &r << endl;
```



Example of use of address-of:

{ ../RIF_POINT/address_1.cc}
{ ../RIF_POINT/rifVsAddressof_0.cc}
{ ../RIF_POINT/rifVsAddressof.cc}

x → 10
&x → 0x7fffeaff4be4

n → 10
&n → 0x7fffeaff57c2

r → 10
&r → 0x7fffeaff57c2
&n → 0x7fffeaff57c2

The operator *address-of «&» (2)*

- The operator “&” (address-of) returns the address (I-value) of the expression where it is applied
- It can be applied only to expression with an address
- It is different with respect to the use of “&” in the definition of references
- Typically, an address requires 4 bytes

Example of use of address-of:
{ ../RIF_POINT/rifVsAddressof_1.cc}
{ ../RIF_POINT/rifVsAddressof.cc}

Note: due to some automatic conversion of the cout stream and some alignment in the management of the different size of the types (char is 8 bits), to print the address of a char variable, you have to use "*(void*)*", example:

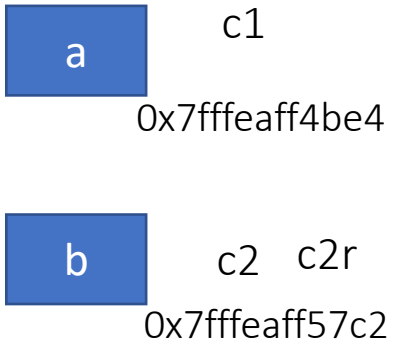
```
char c='a';
cout << &c;  //this gives 'a' … and potentially spurious data (there is no the null character)
cout << (void *)&c;  //it gives the address of c
```

The operator *address-of «&» (2)*

- The operator “&” (address-of) returns the address (I-value) of the expression where it is applied
- It can be applied only to expression with an address
- It is different with respect to the use of “&” in the definition of references
- Typically, an address requires 4 bytes

Example

```
char c1='a';
cout << (void*)&c1 << endl;    // it prints the address of x (not its content)
char c2 = 'b';
char &c2r = c2;          // c2r is the alias of c2
cout << "&c2 = " << (void*)&c2 << ", &c2r = " << (void*)&c2r << endl;
```



Example of use of address-of:
{ ../RIF_POINT/rifVsAddressof_1.cc}
{ ../RIF_POINT/rifVsAddressof.cc}

Note: due to some automatic conversion of the cout stream and some alignment in the management of the different size of the types (char is 8 bits), to print the address of a char variable, you have to use "*(void*)*", example:

```
char c='a';
cout << &c;  //this gives 'a' … and potentially spurious data (there is no the null character)
cout << (void *)&c;  //it gives the address of c
```

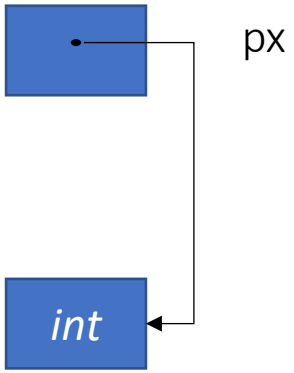
c1 → a
&c1 → 0x7fffeaff4be4

c2→ b
&c2 → 0x7fffeaff57c2

c2r → b
&c2r → 0x7fffeaff57c2
&c2 → 0x7fffeaff57c2

The type *Pointer-to*

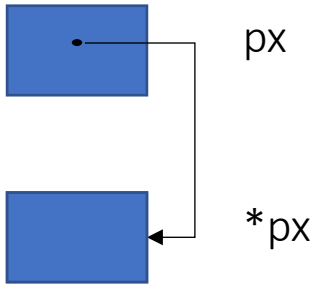
- A pointer contains the address of an object
  - r-value of a pointer is an address
- Syntax: type *id = init;
  - E.g., int *px; //px is a pointer to an integer
  - It is always needed to define the type of the object to which it points
  - The initialization is not mandatory



- A pointer of type T can contain only the address of objects of type T
- A memory space that can contain a memory address is associated to a variable of type pointer
  - ... but no memory space is reserved for the pointed object
- The space allocated to a variable of type pointer is always the same, regardless of the type of object pointed to (typically 8 bytes – with exceptions)

Operator Derefence '*' (1)

- To access to the object pointed by a variable of type pointer, the derefence operator needs to be used
  → '*' allows to modify the pointed object
- If px points to x, *px is a temporary synonym of x
  → by modifying *px, x is modified (and vice-versa)
- *px is an expression with an address

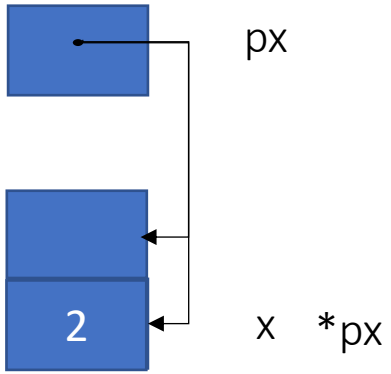


Example

```
int *px;        //px is a pointer variable to a type int
//px=2;         //wrong!
*px=2;          //it writes 2 to the memory cell pointed by the pointer variable
```

Operator Derefence '*' (2)

- To access to the object pointed by a variable of type pointer, the derefence operator needs to be used
  → '*' allows to modify the pointed object
- If px points to x, *px is a temporary synonym of x
  → by modifying *px, x is modified (and vice-versa)
- *px is an expression with an address



Example

```
int x=1;        //x is of type int
int *px;        //px is a pointer variable to a type int
px=&x;          //it access to the variable pointer: &x returns the address of x (not the content of x)
*px=x+1;        //it access to the memory cell pointed by the pointer variable: the value of x+1 is stored in *px
```

Example above:

{ ../RIF_POINT/pointer.cc}

Assignments among Pointers

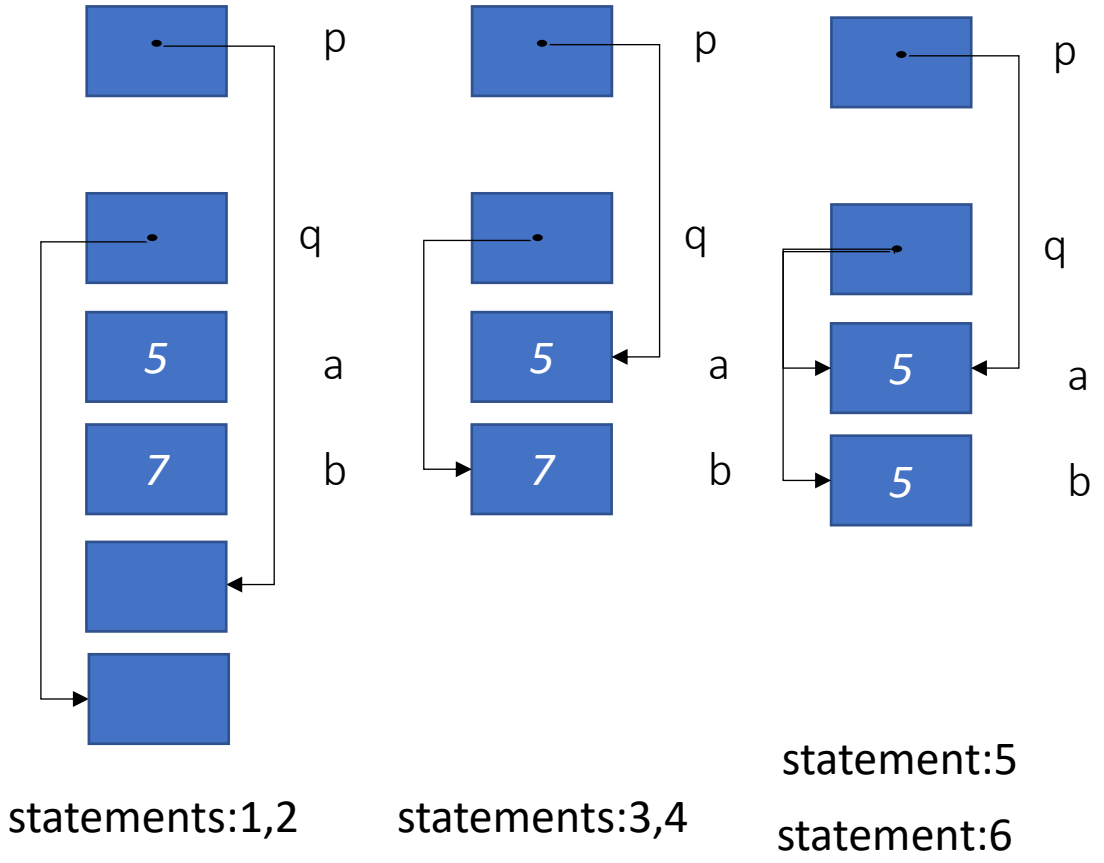
- By assigning to a pointer q the value of another pointer p, q will point to the same object already pointed by p
- *p, *q and the object pointed by them are *temporary synonyms*

Example

```
1.  int a=5, b=7;
2.  int *p, *q;
3.  p= &a;     // p=address of a, *p synonym of a
4.  q= &b;     // q=address of b, *q synonym of b
5.  *q= *p;    // it is equivalent to b=a
6.  q= p;      // it is equivalent to q= address of a
```

Example above:

{ ../RIF_POINT/pointer1.cc}
{ ../RIF_POINT/pointer2.cc}



Examples of pointers and references (*1*)

Example: the reference to an object pointed by a pointer:

Is the reference “following” the pointer?

{ ../RIF_POINT/**rif_deref**.cc} { ../RIF_POINT/rif_deref_0.cc}

```
1.  int n=5;
2.  int *p=&n;
3.  int& r=*p;
4.  int m = 2;
5.  cout << " p = " << p << " &n = " << &n << endl;
6.  cout << "&r = " << &r << " r = " << r <<endl;
7.  cout <<  "&p = " << &p <<endl;
8.  p = &m;
9.  cout << "&m = " << &m << " m = " << m << endl;
10. cout << " p = " << p << " *p = " << *p << endl;
11. cout << "&r = " << &r << " r = " << r <<endl;
12. cout << "&p = " << &p <<endl;
```

Examples of pointers and references (*int*)

Example: the reference to an object pointed by a pointer:

Is the reference “following” the pointer?

{ ../RIF_POINT/rif_deref.cc} { ../RIF_POINT/rif_deref_0.cc}

```
1.  int n=5;
2.  int *p=&n;
3.  int& r=*p;
4.  int m = 2;
5.  cout << " p = " << p << " &n = " << &n << endl;
6.  cout << "&r = " << &r << " r = " << r <<endl;
7.  cout <<  "&p = " << &p <<endl;
8.  p = &m;
9.  cout << "&m = " << &m << " m = " << m << endl;
10. cout << " p = " << p << " *p = " << *p << endl;
11. cout << "&r = " << &r << " r = " << r <<endl;
12. cout << "&p = " << &p <<endl;
```



Examples of pointers and references (*int*)

Example: the reference to an object pointed by a pointer:

Is the reference “following” the pointer?

{ ../RIF_POINT/rif_deref.cc} { ../RIF_POINT/rif_deref_0.cc}

```
1.  int n=5;
2.  int *p=&n;
3.  int& r=*p;
4.  int m = 2;
5.  cout << " p = " << p << " &n = " << &n << endl;
6.  cout << "&r = " << &r << " r = " << r <<endl;
7.  cout <<  "&p = " << &p <<endl;
8.  p = &m;
9.  cout << "&m = " << &m << " m = " << m << endl;
10. cout << " p = " << p << " *p = " << *p << endl;
11. cout << "&r = " << &r << " r = " << r <<endl;
12. cout << "&p = " << &p <<endl;
```



Examples of pointers and references (*int*)

Example: the reference to an object pointed by a pointer:

Is the reference “following” the pointer?

{ ../RIF_POINT/rif_deref.cc} { ../RIF_POINT/rif_deref_0.cc}

```
1.  int n=5;
2.  int *p=&n;
3.  int& r=*p;
4.  int m = 2;
5.  cout << " p = " << p << " &n = " << &n << endl;
6.  cout << "&r = " << &r << " r = " << r <<endl;
7.  cout <<  "&p = " << &p <<endl;
8.  p = &m;
9.  cout << "&m = " << &m << " m = " << m << endl;
10. cout << " p = " << p << " *p = " << *p << endl;
11. cout << "&r = " << &r << " r = " << r <<endl;
12. cout << "&p = " << &p <<endl;
```



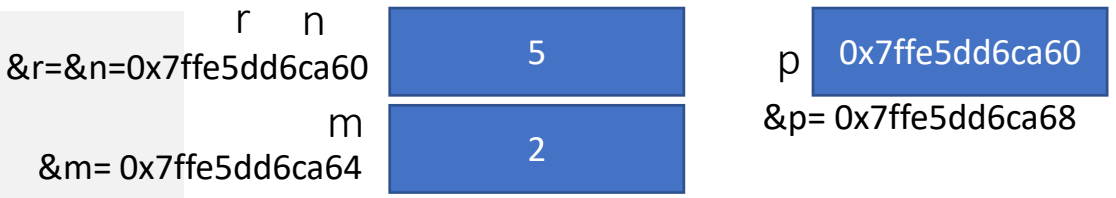
Examples of pointers and references (*int*)

Example: the reference to an object pointed by a pointer:

Is the reference “following” the pointer?

{ ../RIF_POINT/rif_deref.cc} { ../RIF_POINT/rif_deref_0.cc}

```
1.  int n=5;
2.  int *p=&n;
3.  int& r=*p;
4.  int m = 2;
5.  cout << " p = " << p << " &n = " << &n << endl;
6.  cout << "&r = " << &r << " r = " << r <<endl;
7.  cout <<  "&p = " << &p <<endl;
8.  p = &m;
9.  cout << "&m = " << &m << " m = " << m << endl;
10. cout << " p = " << p << " *p = " << *p << endl;
11. cout << "&r = " << &r << " r = " << r <<endl;
12. cout << "&p = " << &p <<endl;
```



Examples of pointers and references (*int*)

Example: the reference to an object pointed by a pointer:

Is the reference “following” the pointer?

{ ../RIF_POINT/rif_deref.cc} { ../RIF_POINT/rif_deref_0.cc}

```
1.  int n=5;
2.  int *p=&n;
3.  int& r=*p;
4.  int m = 2;
5.  cout << " p = " << p << " &n = " << &n << endl;
6.  cout << "&r = " << &r << " r = " << r <<endl;
7.  cout <<  "&p = " << &p <<endl;
8.  p = &m;
9.  cout << "&m = " << &m << " m = " << m << endl;
10. cout << " p = " << p << " *p = " << *p << endl;
11. cout << "&r = " << &r << " r = " << r <<endl;
12. cout << "&p = " << &p <<endl;
```



Examples of pointers and references (*int*)

Example: the reference to an object pointed by a pointer:

Is the reference “following” the pointer?

{ ../RIF_POINT/rif_deref.cc} { ../RIF_POINT/rif_deref_0.cc}

```
1.  int n=5;
2.  int *p=&n;
3.  int& r=*p;
4.  int m = 2;
5.  cout << " p = " << p << " &n = " << &n << endl;
6.  cout << "&r = " << &r << " r = " << r <<endl;
7.  cout <<  "&p = " << &p <<endl;
8.  p = &m;
9.  cout << "&m = " << &m << " m = " << m << endl;
10. cout << " p = " << p << " *p = " << *p << endl;
11. cout << "&r = " << &r << " r = " << r <<endl;
12. cout << "&p = " << &p <<endl;
```



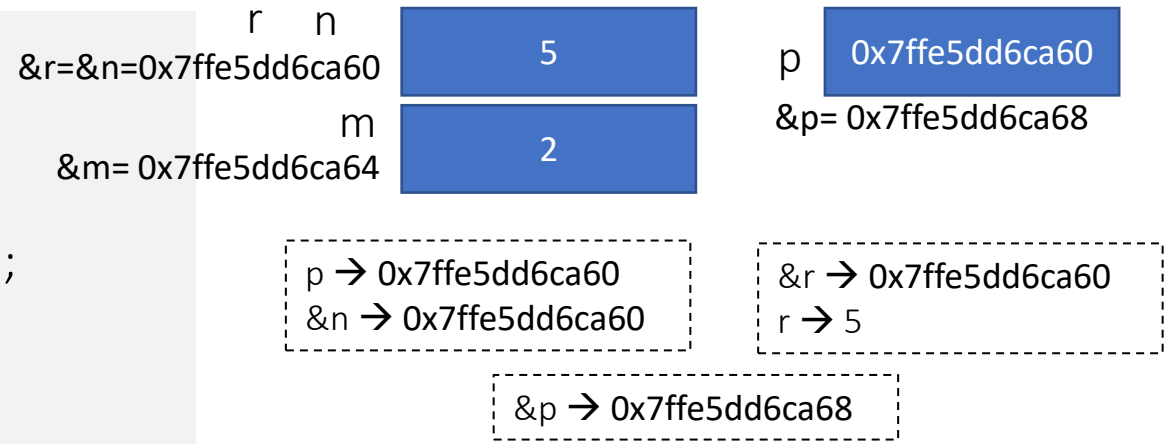
Examples of pointers and references (*int*)

Example: the reference to an object pointed by a pointer:

Is the reference “following” the pointer?

{ ../RIF_POINT/rif_deref.cc} { ../RIF_POINT/rif_deref_0.cc}

```
1.  int n=5;
2.  int *p=&n;
3.  int& r=*p;
4.  int m = 2;
5.  cout << " p = " << p << " &n = " << &n << endl;
6.  cout << "&r = " << &r << " r = " << r <<endl;
7.  cout <<  "&p = " << &p <<endl;
8.  p = &m;
9.  cout << "&m = " << &m << " m = " << m << endl;
10. cout << " p = " << p << " *p = " << *p << endl;
11. cout << "&r = " << &r << " r = " << r <<endl;
12. cout << "&p = " << &p <<endl;
```



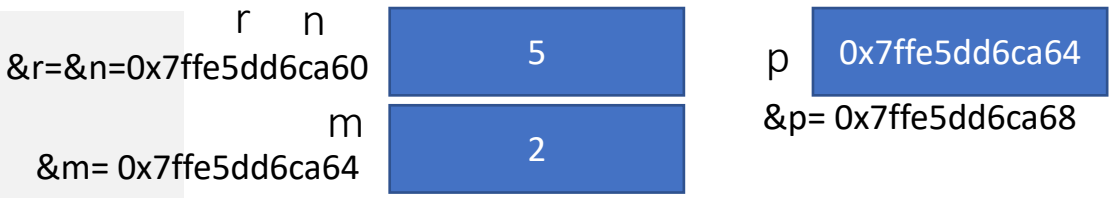
Examples of pointers and references (*int*)

Example: the reference to an object pointed by a pointer:

Is the reference “following” the pointer?

{ ../RIF_POINT/rif_deref.cc} { ../RIF_POINT/rif_deref_0.cc}

```
1.  int n=5;
2.  int *p=&n;
3.  int& r=*p;
4.  int m = 2;
5.  cout << " p = " << p << " &n = " << &n << endl;
6.  cout << "&r = " << &r << " r = " << r <<endl;
7.  cout << "&p = " << &p <<endl;
8.  p = &m;
9.  cout << "&m = " << &m << " m = " << m << endl;
10. cout << " p = " << p << " *p = " << *p << endl;
11. cout << "&r = " << &r << " r = " << r <<endl;
12. cout << "&p = " << &p <<endl;
```



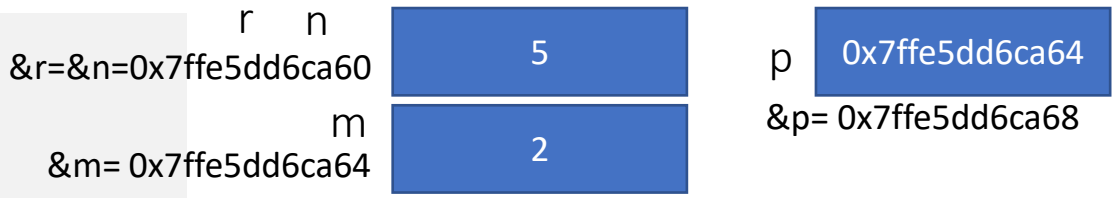
Examples of pointers and references (*int*)

Example: the reference to an object pointed by a pointer:

Is the reference “following” the pointer?

{ ../RIF_POINT/rif_deref.cc} { ../RIF_POINT/rif_deref_0.cc}

```
1.  int n=5;
2.  int *p=&n;
3.  int& r=*p;
4.  int m = 2;
5.  cout << " p = " << p << " &n = " << &n << endl;
6.  cout << "&r = " << &r << " r = " << r <<endl;
7.  cout << "&p = " << &p <<endl;
8.  p = &m;
9.  cout << "&m = " << &m << " m = " << m << endl;
10. cout << " p = " << p << " *p = " << *p << endl;
11. cout << "&r = " << &r << " r = " << r <<endl;
12. cout << "&p = " << &p <<endl;
```



&m → 0x7ffe5dd6ca64
m → 2

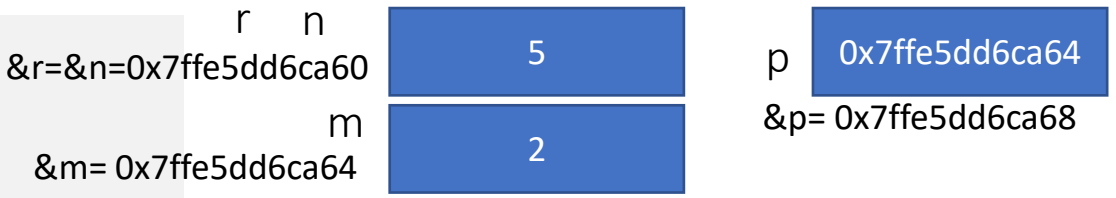
Examples of pointers and references (*int*)

Example: the reference to an object pointed by a pointer:

Is the reference “following” the pointer?

{ ../RIF_POINT/rif_deref.cc} { ../RIF_POINT/rif_deref_0.cc}

```
1.  int n=5;
2.  int *p=&n;
3.  int& r=*p;
4.  int m = 2;
5.  cout << " p = " << p << " &n = " << &n << endl;
6.  cout << "&r = " << &r << " r = " << r <<endl;
7.  cout << "&p = " << &p <<endl;
8.  p = &m;
9.  cout << "&m = " << &m << " m = " << m << endl;
10. cout << " p = " << p << " *p = " << *p << endl;
11. cout << "&r = " << &r << " r = " << r <<endl;
12. cout << "&p = " << &p <<endl;
```



&m → 0x7ffe5dd6ca64
m → 2

p → 0x7ffe5dd6ca64
*p → 2

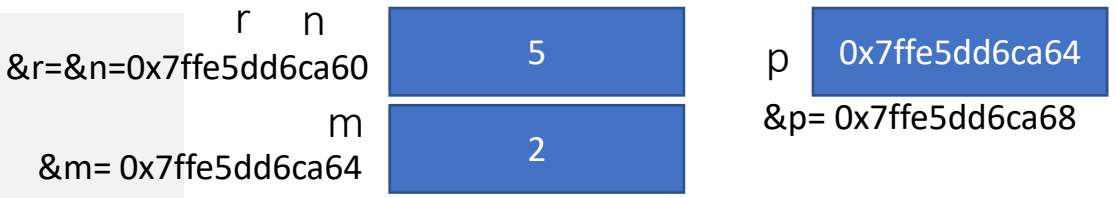
Examples of pointers and references (*int*)

Example: the reference to an object pointed by a pointer:

Is the reference “following” the pointer?

{ ../RIF_POINT/rif_deref.cc} { ../RIF_POINT/rif_deref_0.cc}

```
1.  int n=5;
2.  int *p=&n;
3.  int& r=*p;
4.  int m = 2;
5.  cout << " p = " << p << " &n = " << &n << endl;
6.  cout << "&r = " << &r << " r = " << r <<endl;
7.  cout << "&p = " << &p <<endl;
8.  p = &m;
9.  cout << "&m = " << &m << " m = " << m << endl;
10. cout << " p = " << p << " *p = " << *p << endl;
11. cout << "&r = " << &r << " r = " << r <<endl;
12. cout << "&p = " << &p <<endl;
```



&m → 0x7ffe5dd6ca64
m → 2

p → 0x7ffe5dd6ca64
*p → 2

&r → 0x7ffe5dd6ca60
r → 5

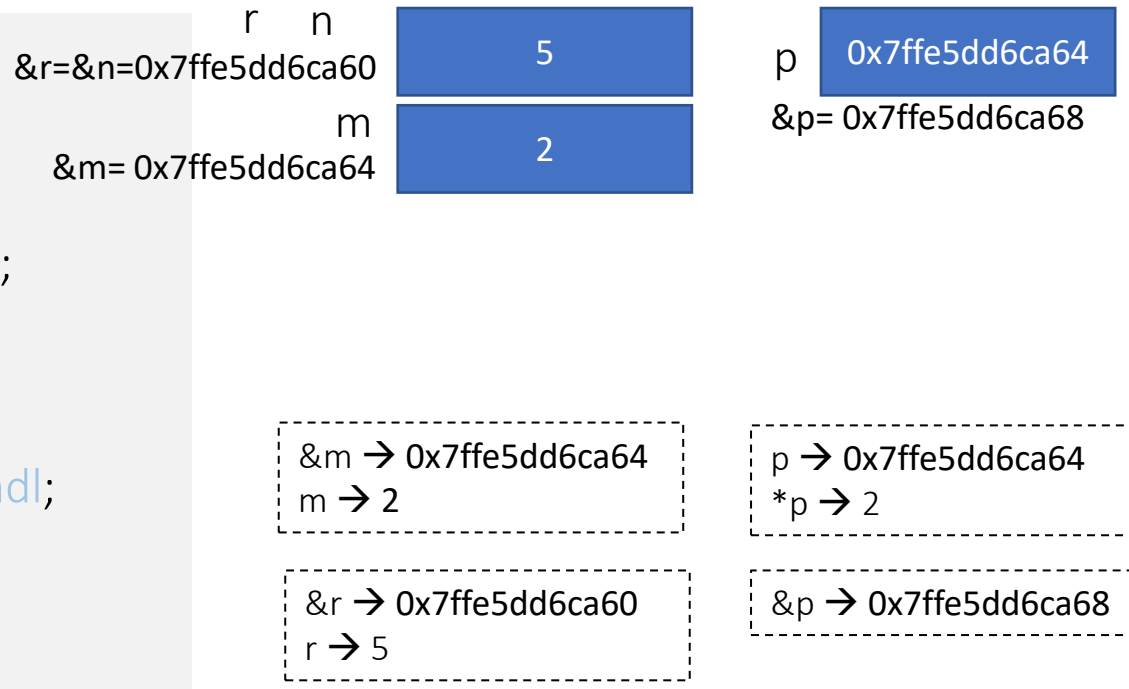
Examples of pointers and references (*int*)

Example: the reference to an object pointed by a pointer:

Is the reference “following” the pointer?

{ ../RIF_POINT/rif_deref.cc} { ../RIF_POINT/rif_deref_0.cc}

```
1.  int n=5;
2.  int *p=&n;
3.  int& r=*p;
4.  int m = 2;
5.  cout << " p = " << p << " &n = " << &n << endl;
6.  cout << "&r = " << &r << " r = " << r <<endl;
7.  cout << "&p = " << &p <<endl;
8.  p = &m;
9.  cout << "&m = " << &m << " m = " << m << endl;
10. cout << " p = " << p << " *p = " << *p << endl;
11. cout << "&r = " << &r << " r = " << r <<endl;
12. cout << "&p = " << &p <<endl;
```



Line 8: we moved the pointer *p* to point *m*, but the alias *r* was not moved

Examples of pointers and references (*2*)

Example with char: the reference to an object pointed by a pointer:

Is the reference “following” the pointer?

{ ../RIF_POINT/rif_deref_2.cc}

```
1.  char c='a';          //&c = 0x7ffe5dd6c626
2.  char *p=&c;
3.  char& r=*p;
4.  char m = 'b';    //&n = 0x7ffe5dd6c627
5.  cout << " p = " << (void*)p << " &c = " << (void*)&c << endl;
6.  cout << " *p=" << *p << " c=" << c << endl;
7.  cout << " &p=" << (void*)p << endl;
8.  cout << "&r = " << (void*)&r << " r = " << r <<endl;
9.  p = &m;
10. cout << "&m = " << (void*)&m << " m = " << m << endl;
11. cout << " p = " << (void*)p << " *p = " << *p << endl;
12. cout << "&r = " << (void*)&r << " r = " << r <<endl;
13. cout << " &p=" << (void*)&p << endl;
```

RECALL: Variant with char: due to some automatic conversion of the cout stream and some alignment in the management of the different size of the types (char is 8 bits), to print the address of a char variable, you have to use "(void*)", example:

```
char c='a';
cout << &c;  //this gives a… and potentially spurious data (there is no the null character)
cout << (void *)&c; //ok, it gives the address of c
```

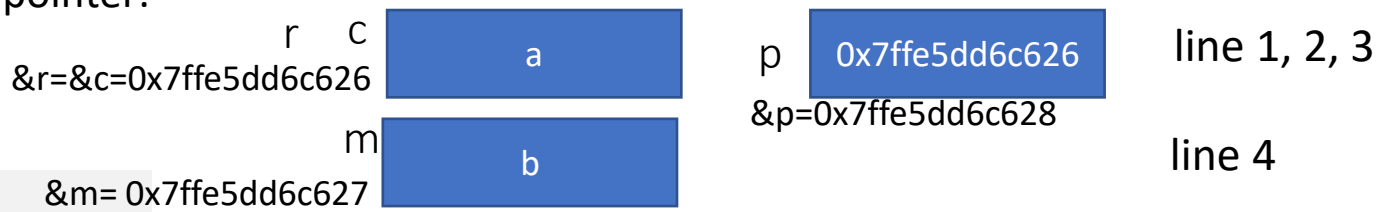
Examples of pointers and references (char)

Example with char: the reference to an object pointed by a pointer:

Is the reference “following” the pointer?

{ ../RIF_POINT/rif_deref_2.cc}

```
1.   char c='a';          //&c = 0x7ffe5dd6c626
2.   char *p=&c;
3.   char& r=*p;
4.   char m = 'b';    //&n = 0x7ffe5dd6c627
5.   cout << " p = " << (void*)p << " &c = " << (void*)&c << endl;
6.   cout << " *p=" << *p << " c=" << c << endl;
7.   cout << " &p=" << (void*)&p << endl;
8.   cout << "&r = " << (void*)&r << " r = " << r <<endl;
9.   p = &m;
10.  cout << "&m = " << (void*)&m << " m = " << m << endl;
11.  cout << " p = " << (void*)p << " *p = " << *p << endl;
12.  cout << "&r = " << (void*)&r << " r = " << r <<endl;
13.  cout << " &p=" << (void*)&p << endl;
```



line 1, 2, 3

line 4

RECALL: Variant with char: due to some automatic conversion of the cout stream and some alignment in the management of the different size of the types (char is 8 bits), to print the address of a char variable, you have to use "(void*)", example:

```
char c='a';
cout << &c;  //this gives a… and potentially spurious data (there is no the null character)
cout << (void *)&c; //ok, it gives the address of c
```

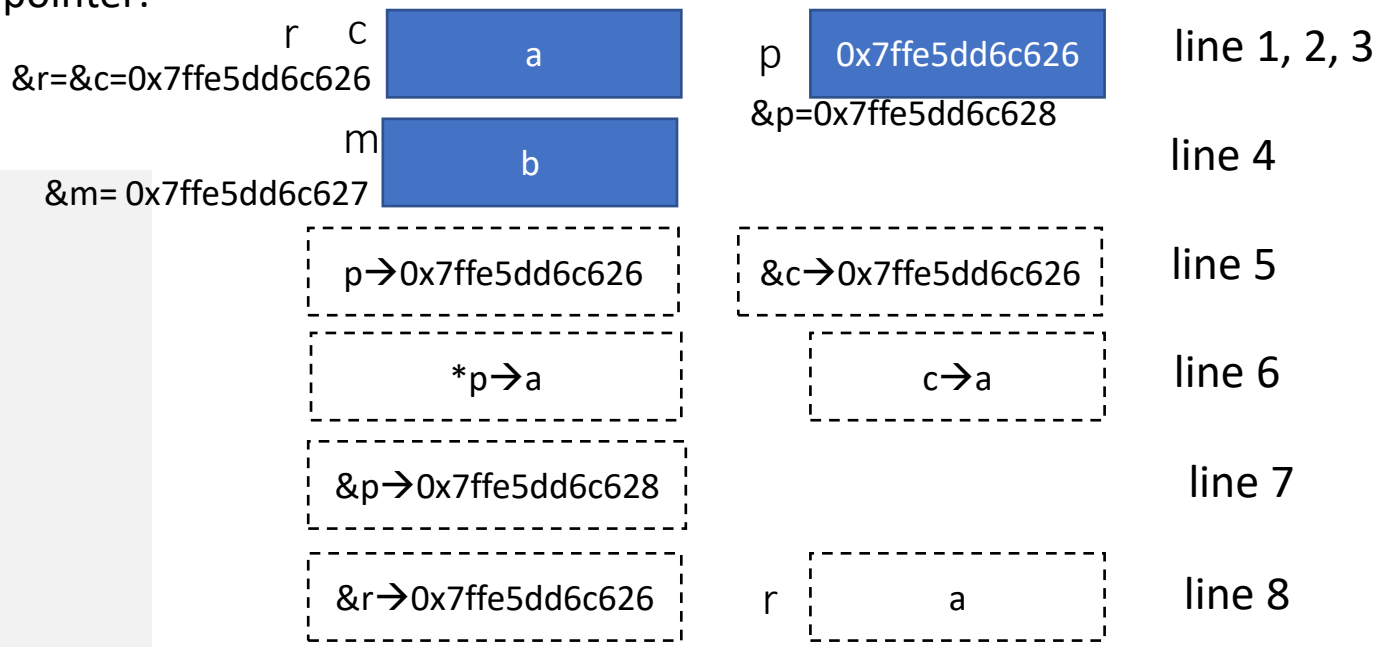
Examples of pointers and references (char)

Example with char: the reference to an object pointed by a pointer:

Is the reference “following” the pointer?

{ ../RIF_POINT/rif_deref_2.cc}

```
1.  char c='a';          //&c = 0x7ffe5dd6c626
2.  char *p=&c;
3.  char& r=*p;
4.  char m = 'b';    //&n = 0x7ffe5dd6c627
5.  cout << " p = " << (void*)p << " &c = " << (void*)&c << endl;
6.  cout << " *p=" << *p << " c=" << c << endl;
7.  cout << " &p=" << (void*)&p << endl;
8.  cout << "&r = " << (void*)&r << " r = " << r <<endl;
9.  p = &m;
10. cout << "&m = " << (void*)&m << " m = " << m << endl;
11. cout << " p = " << (void*)p << " *p = " << *p << endl;
12. cout << "&r = " << (void*)&r << " r = " << r <<endl;
13. cout << " &p=" << (void*)&p << endl;
```



RECALL: Variant with char: due to some automatic conversion of the cout stream and some alignment in the management of the different size of the types (char is 8 bits), to print the address of a char variable, you have to use "(void*)", example:

```
char c='a';
cout << &c;  //this gives a… and potentially spurious data (there is no the null character)
cout << (void *)&c; //ok, it gives the address of c
```

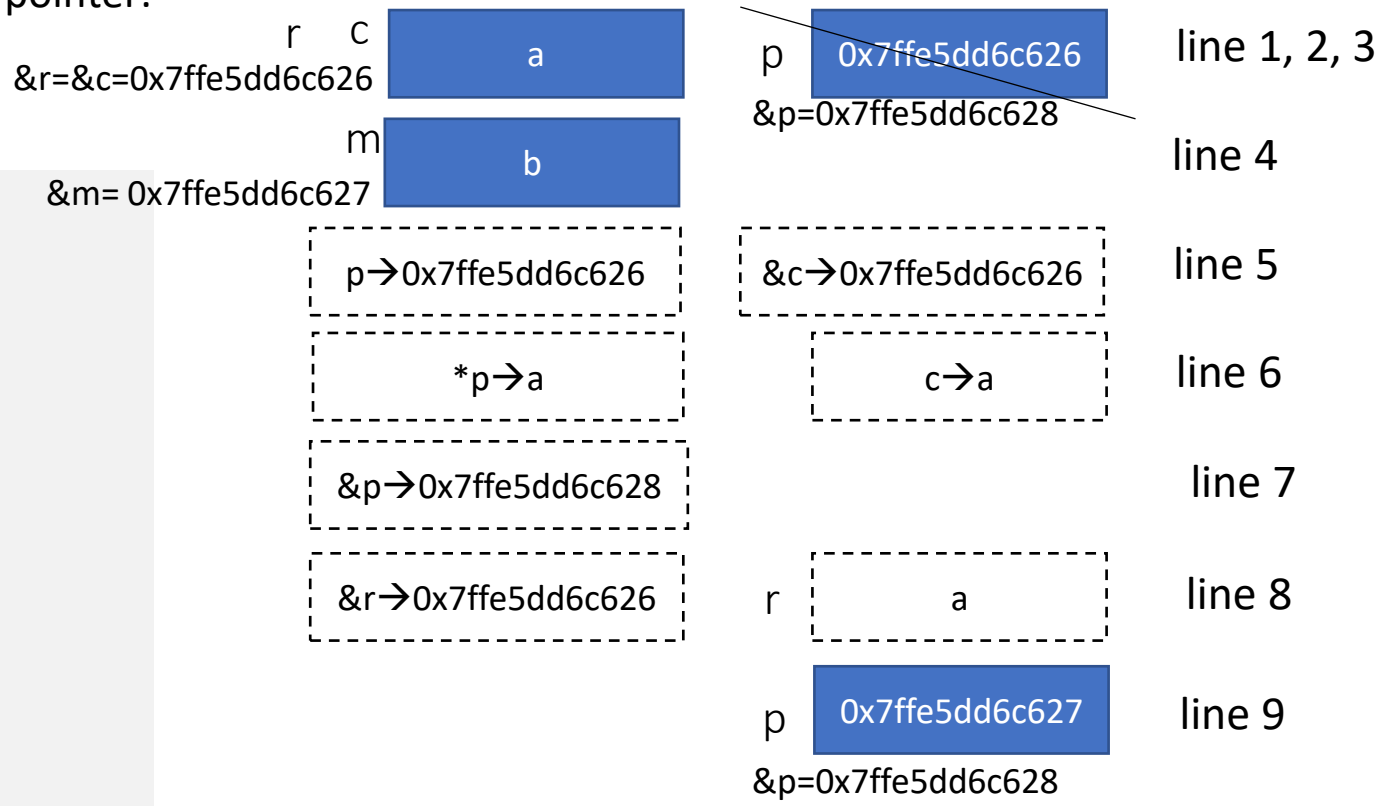
Examples of pointers and references (char)

Example with char: the reference to an object pointed by a pointer:

Is the reference “following” the pointer?

{ ../RIF_POINT/rif_deref_2.cc}

```
1.  char c='a';          //&c = 0x7ffe5dd6c626
2.  char *p=&c;
3.  char& r=*p;
4.  char m = 'b';    //&n = 0x7ffe5dd6c627
5.  cout << " p = " << (void*)p << " &c = " << (void*)&c << endl;
6.  cout << " *p=" << *p << " c=" << c << endl;
7.  cout << " &p=" << (void*)&p << endl;
8.  cout << "&r = " << (void*)&r << " r = " << r <<endl;
9.  p = &m;
10. cout << "&m = " << (void*)&m << " m = " << m << endl;
11. cout << " p = " << (void*)p << " *p = " << *p << endl;
12. cout << "&r = " << (void*)&r << " r = " << r <<endl;
13. cout << " &p=" << (void*)&p << endl;
```



RECALL: Variant with char: due to some automatic conversion of the cout stream and some alignment in the management of the different size of the types (char is 8 bits), to print the address of a char variable, you have to use "(void*)", example:

```
char c='a';
cout << &c;  //this gives a… and potentially spurious data (there is no the null character)
cout << (void *)&c; //ok, it gives the address of c
```

Examples of pointers and references (char)

Example with char: the reference to an object pointed by a pointer:

Is the reference “following” the pointer?

{ ../RIF_POINT/rif_deref_2.cc}

```
1.  char c='a';          //&c = 0x7ffe5dd6c626
2.  char *p=&c;
3.  char& r=*p;
4.  char m = 'b';    //&n = 0x7ffe5dd6c627
5.  cout << " p = " << (void*)p << " &c = " << (void*)&c << endl;
6.  cout << " *p=" << *p << " c=" << c << endl;
7.  cout << " &p=" << (void*)&p << endl;
8.  cout << "&r = " << (void*)&r << " r = " << r <<endl;
9.  p = &m;
10. cout << "&m = " << (void*)&m << " m = " << m << endl;
11. cout << " p = " << (void*)p << " *p = " << *p << endl;
12. cout << "&r = " << (void*)&r << " r = " << r <<endl;
13. cout << " &p=" << (void*)&p << endl;
```

RECALL: Variant with char: due to some automatic conversion of the cout stream and some alignment in the management of the different size of the types (char is 8 bits), to print the address of a char variable, you have to use "(void*)", example:

```
char c='a';
cout << &c;  //this gives a… and potentially spurious data (there is no the null character)
cout << (void *)&c; //ok, it gives the address of c
```



Pointers to *void*

- In some cases, it could be useful to have a variable that can point to an entity of a different type
- This variable is declared of type “pointer to void”, i.e., not specified type

Example

```
1.  int a;
2.  int *pa=&a;                 //the address of a is assigned to the content pointed by pa
3.  char c;
4.  char *pc=&c;                //the address of c is assigned to the content pointed by pc
5.  void *tp;                   //declaration of a not specified type
6.
7.  tp=pa;                      //tp points to the same variable of type int pointed by pa (-> a)
8.  *(int*)tp=3;                //set to 3 the content of tp (typed with int) (-> a = 3)
9.
10. tp= pc                      //tp points to the same variable of type char pointed by pc (-> c)
11. *(char*)tp='C';             //set to 'C' the content of tp (typed with char) (-> c = 'C')
```

Example above:

{ ../RIF_POINT/point_to_void.cc}
{ ../RIF_POINT/point3.cc}

Point to void: example

```
1. using namespace std;
2. #include <iostream>
3.
4. int main ()
5. {
6.   int a=4;
7.   int *pa=&a;          //pointer to int
8.   char c;
9.   char *pc=&c;         // pointer to char
10.  void *tp;            // declaration of a pointer to void
11.  tp = pa;             // tp points same as pa (-> int a)
12.  *(int*)tp=3;         // the content (int) of tp is set to 3
13.  cout << "(1) *tp = " << *(int*)tp << endl;  // content of (int)tp is 3
14.  cout << "(2) *pa = " << *pa << endl;  //the content of (int)pa is 3
15.  cout << "(3) a = " << a << endl; //the content of a is 3
16.  tp = pc;             // tp points same as pc (-> char c)
17.  *(char*)tp='C';      // content of (char) tp is 'C'
18.  cout << "(4) *tp = " << *(char*)tp << endl;  // content of tp (char) is 'C'
19.
20. return 0;
21.}
```

```
$ g++ point_to_void.cc
$ ./a.out
(1) *tp = 3
(2) *pa = 3
(3) a = 3
(4) *tp = C
```


{ ../RIF_POINT/point_to_void.cc}

Pointers to *constant (1)*

Syntax

const type *id = init;

E.g.,: const int *pc1 = &c1;

- Intuition: they do not allow to modify the pointed object via *derefence* of the same pointer

Note: the pointer object is not transformed in a constant

Example

```
1.  const int c1 = 3;          //c1=4; is wrong: compile-time error, c1 is a constant variable
2.  int c2 = 5;
3.  const int *pc1 = &c1;      // ok *pc1=3
4.  const int *pc2 = &c2;      // ok  *pc2=5
5.  //int *pc3= &c1;           // wrong: compile-time error
6.  //*pc1 = 2;                // wrong: compile-time error
7.  //*pc2 = 2;                // wrong: compile-time error
8.  pc2 = pc1;                 // ok *pc2=3
9.  pc1 = &c2;                 // ok *pc1=5
10. //*pc1 = 2;                // wrong: compile-time error
11. //*pc2 = 2;                // wrong: compile-time error
12. c2 = 2;                    // ok
```

Example above:

{ ../RIF_POINT/point_to_const.cc}

Pointers to *constant (2)*

Syntax

const type *id = init;

E.g.,: const int *pc1 = &c1;

- Intuition: they do not allow to modify the pointed object via *derefence* of the same pointer

Note: the pointer object is not transformed in a constant

Example

```
1.  const int c1 = 3;
2.  int c2 = 5;
3.  const int *pc1 = &c1;        // ok *pc1=3
4.  const int *pc2 = &c2;        // ok  *pc2=5
5.  //int *pc3= &c1;             // wrong: compile-time error
6.  pc2 = pc1;                   // ok *pc2=3
7.  pc1 = &c2;                   // ok *pc1=5
8.  //*pc1 = 2;                  // wrong: compile-time error
9.  c2 = 2;                      // ok
```

Line (after the execution of the line..)	c1	c2	*pc1	*pc2
1	3			
2	3	5		
3	3	5	3	
4	3	5	3	5
6	3	5	3	3
7	3	5	5	3
9	3	2	2	3

Example above:

{ ../RIF_POINT/point_to_const.cc}

Constants pointer

- Syntax
  type *const id=exp;
  E.g., int *const pa = &a;
- Intuition: they do not allow to point to another object
- The initialization ("=exp") is mandatory

Note: the pointed object can be modified by using *derefence* to the same pointer

Example

```
int a,b;                //it is always better to initialize the variables, but in this case the result is the same
int *const pa = &a;
*pa = 3;                // ok
//pa = &b;              //wrong pa is a constant: compile-time error (pa is read-only)
```

Example above:
{ ../RIF_POINT/const_point.cc}

Constants pointer to constant (1)

- Syntax
  const type *const id=exp;
  E.g., const int *const a = &c;
- Intuition: they do not allow to point to another object and the pointed object cannot be modified by using *derefence* of the same pointer

Example

```
const int b = 2;
const int c = 3;
const int *const a = &c;
//a = &b;          // wrong: cannot change the pointer
//*a= 2;           // wrong: cannot change the pointed object
//c = 5;           // wrong: the variable is constant
```

Example above:
{ ../RIF_POINT/const_point_const.cc}

Constants pointer to constant (2)

- Syntax
  const type *const id=exp;
  E.g., const int *const a = &c;
- Intuition: they do not allow to point to another object and the pointed object cannot be modified by using *derefence* of the same pointer

Example

```
const int b = 2;
const int c = 3;
const int *const a = &c;
//a = &b;          // wrong: cannot change the pointer
//*a= 2;           // wrong: cannot change the pointed object
c = 5;             // the variable c can be changed since it is not a constant variable
```

Example above:
{ ../RIF_POINT/const_point_const.cc}

Pointers to pointers

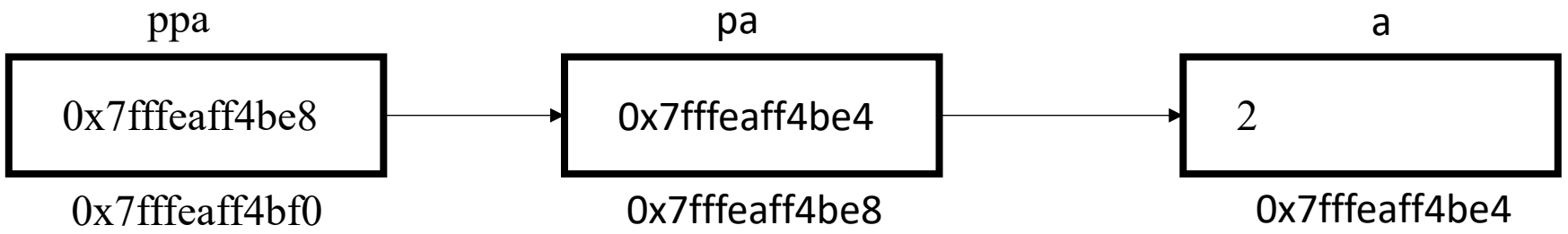
- A pointer variable is a variable with one type (like any other variables), for which it is possible to define pointers to these variables
- The address of a pointer variable is a pointer to a pointer

E.g.,

- `int * p;` // pointer to an integer
- `int **p;` // pointer to a pointer of an integer
- `char **c;` // pointer to a pointer of a char

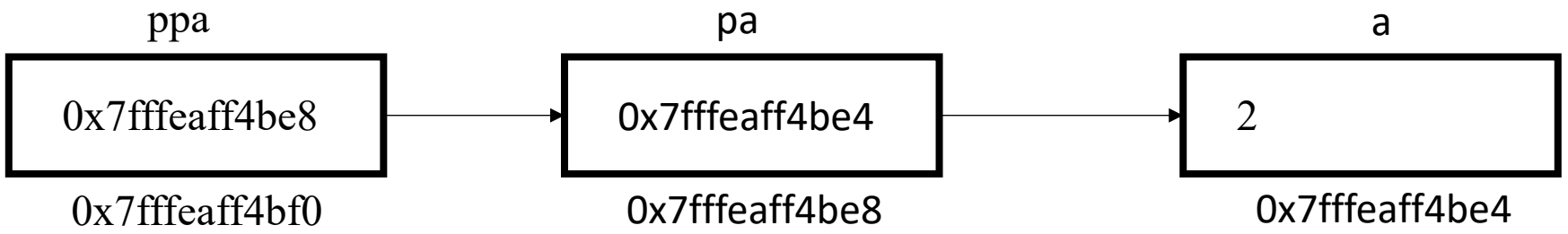
Pointers to pointers (2)

```
1.  int main () {
2.      int a, *pa, ** ppa;
3.      a = 2;
4.      pa = &a;
5.      ppa = &pa;
6.      cout << “Address of a = " << &a;
7.      cout << " value of a = " << a << endl;
8.      cout << "Address of pa = " << &pa;
9.      cout << " value of pa = " << pa << endl;
10.     cout << "Address of ppa = " << &ppa;
11.     cout << " value of ppa = " << ppa << endl;
12. }
```



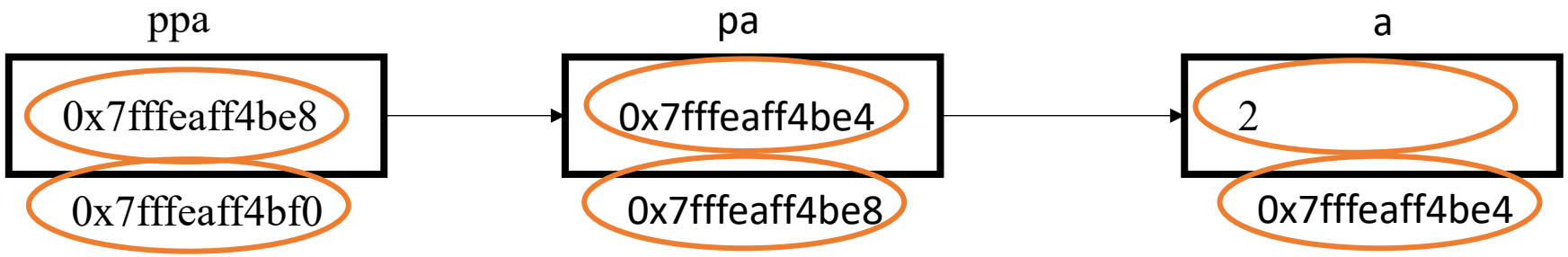
Pointers to pointers (2)

```
1.  int main () {
2.      int a, *pa, ** ppa;
3.      a = 2;
4.      pa = &a;
5.      ppa = &pa;
6.      cout << “Address of a = " << &a;
7.      cout << " value of a = " << a << endl;
8.      cout << "Address of pa = " << &pa;
9.      cout << " value of pa = " << pa << endl;
10.     cout << "Address of ppa = " << &ppa;
11.     cout << " value of ppa = " << ppa << endl;
12. }
```



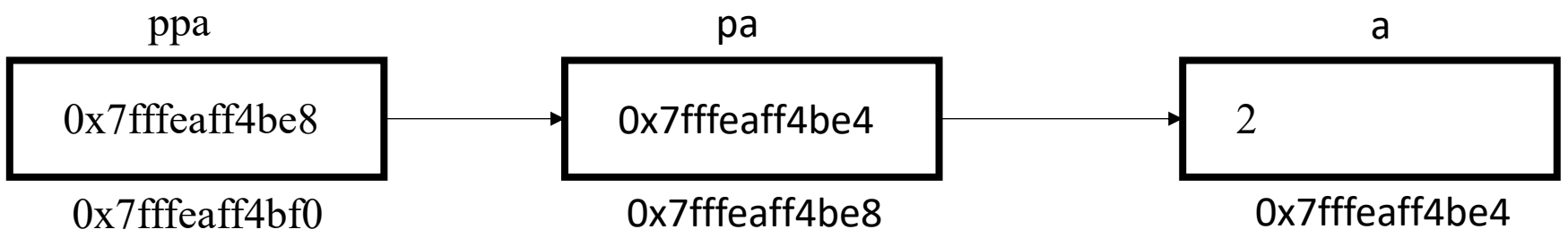
Pointers to pointers (2)

```
1.  int main () {
2.      int a, *pa, ** ppa;
3.      a = 2;
4.      pa = &a;
5.      ppa = &pa;
6.      cout << “Address of a = " << &a;
7.      cout << " value of a = " << a << endl;
8.      cout << "Address of pa = " << &pa;
9.      cout << " value of pa = " << pa << endl;
10.     cout << "Address of ppa = " << &ppa;
11.     cout << " value of ppa = " << ppa << endl;
12. }
```



Pointers to pointers (3)

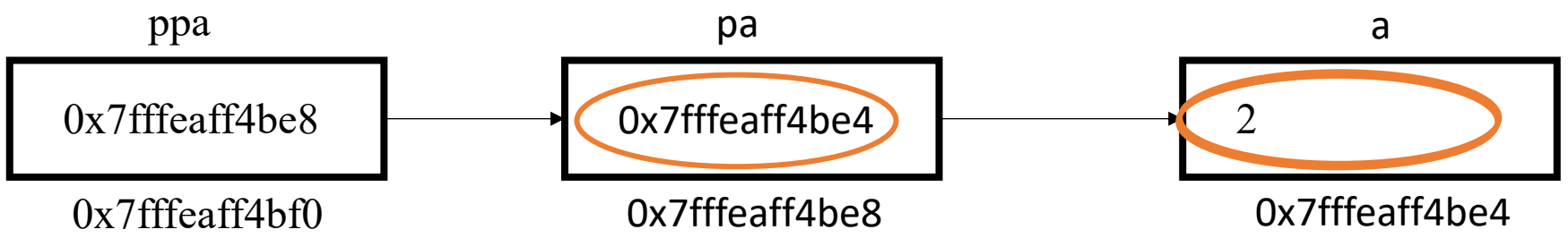
```
1.  int main () {
2.      int a, *pa, ** ppa;
3.      a = 2;
4.      pa = &a;
5.      ppa = &pa;
6.      cout << “Address of a = " << &a;
7.      cout << " value of a = " << a << endl;
8.      cout << "Address of pa = " << &pa;
9.      cout << " value of pa = " << pa << endl;
10.     cout << "Address of ppa = " << &ppa;
11.     cout << " value of ppa = " << ppa << endl;
12.     //
13.     cout << " value of *pa = " << *pa << endl;
14.     cout << " value of *ppa = " << *ppa << endl;
15.     cout << " value of **ppa = " << **ppa << endl;
16. }
```



Example above:
{ ../RIF_POINT/pointers_to_pointers.cc}

Pointers to pointers (3)

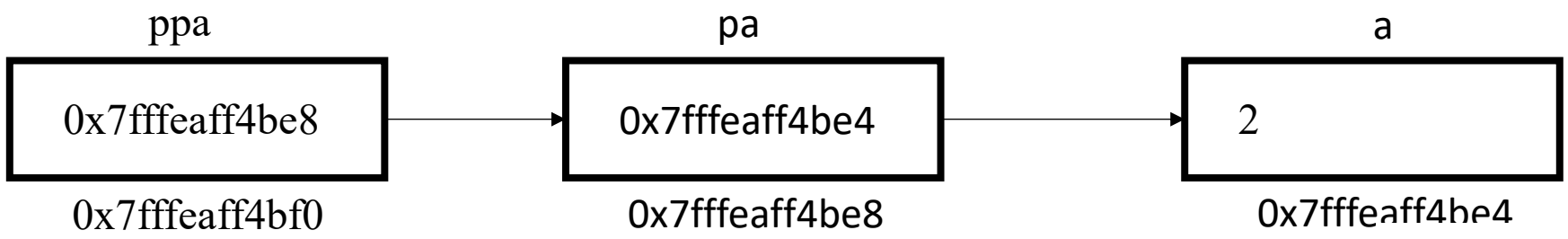
```
1.  int main () {
2.      int a, *pa, ** ppa;
3.      a = 2;
4.      pa = &a;
5.      ppa = &pa;
6.      cout << “Address of a = " << &a;
7.      cout << " value of a = " << a << endl;
8.      cout << "Address of pa = " << &pa;
9.      cout << " value of pa = " << pa << endl;
10.     cout << "Address of ppa = " << &ppa;
11.     cout << " value of ppa = " << ppa << endl;
12.     //
13.     cout << " value of *pa = " << *pa << endl;
14.     cout << " value of *ppa = " << *ppa << endl;
15.     cout << " value of **ppa = " << **ppa << endl;
16. }
```



Example above:
{ ../RIF_POINT/pointers_to_pointers.cc}

Pointers to pointers (3)

```
1.  int main () {
2.      int a, *pa, ** ppa;
3.      a = 2;
4.      pa = &a;
5.      ppa = &pa;
6.      cout << “Address of a = " << &a;                  // 0x7fffeaff4be4
7.      cout << " value of a = " << a << endl;            // 2
8.      cout << "Address of pa = " << &pa;                // 0x7fffeaff4be8
9.      cout << " value of pa = " << pa << endl;          // 0x7fffeaff4be4
10.     cout << "Address of ppa = " << &ppa;              // 0x7fffeaff4bf0
11.     cout << " value of ppa = " << ppa << endl;        // 0x7fffeaff4be8
12.     //
13.     cout << " value of *pa = " << *pa << endl;        // 2
14.     cout << " value of *ppa = " << *ppa << endl;      // 0x7fffeaff4be4
15.     cout << " value of **ppa = " << **ppa << endl;    // 2 ( i.e., we can see as *(*ppa) )
16. }
```



Example above:

{ ../RIF_POINT/pointers_to_pointers.cc}

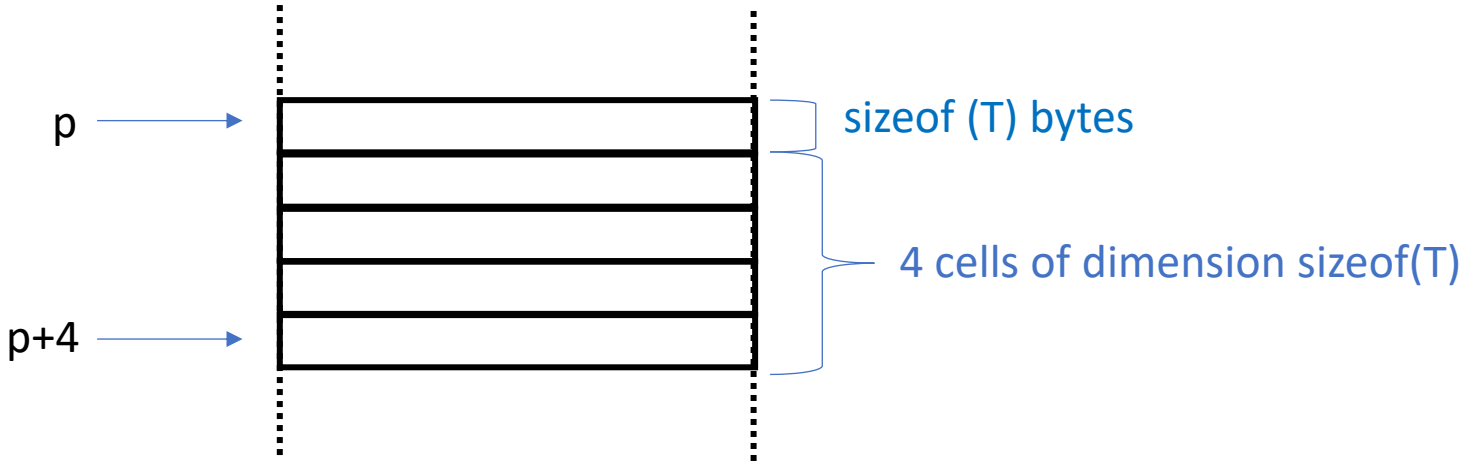
{ ../RIF_POINT/point_point.cc}

Arithmetic's of pointers and addresses (1)

Addresses and pointers have an arithmetic:

- If p is of type T* and index is of type integer, then:
  - p+index is of type T*
  - p+index is the address of an object of type T that is in the memory after index position of dimension sizeof(T)
  - Similary for p++, ++p, p--, --p, p+=index, etc.

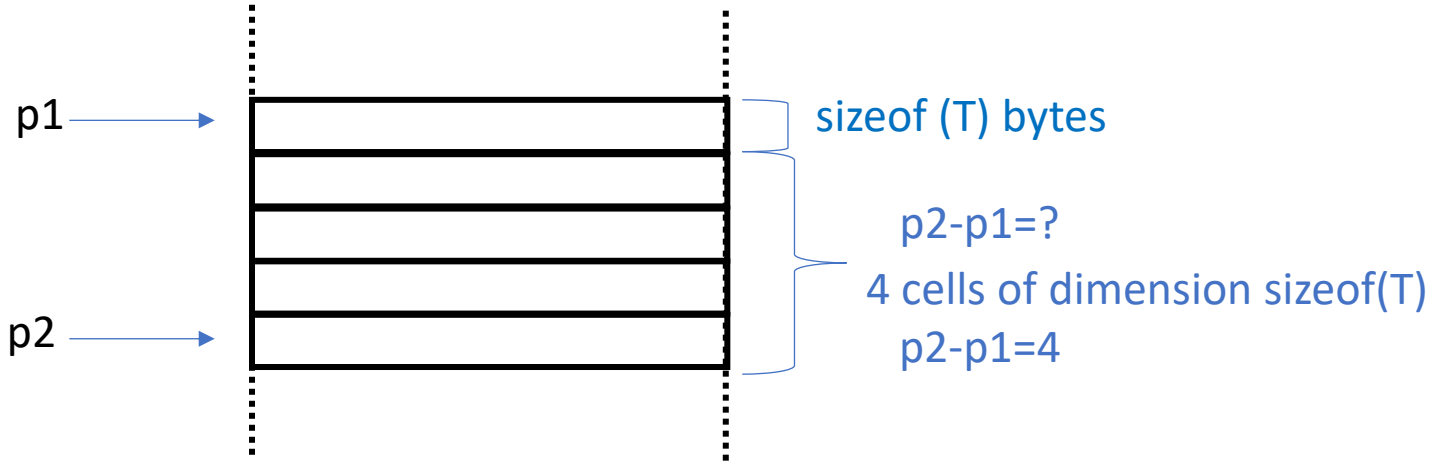
→index is implicitly multiplied by sizeof(T)



Arithmetic's of pointers and addresses (2)

- If p1, p2 are of type T*, then:
  - p2-p1 is an integer
  - p2-p1 is the number of positions of dimension sizeof(T) for which p1 precedes p2 (negative number if p2 precedes p1)
  - it is possible to apply operators of comparison p1<p2, p1>=p2, etc.

→p2-p1 is implicitly dived per sizeof(T)



Example of arithmetic operations over pointers:

{ ../RIF_POINT/arithmetic_point.cc}

Arithmetic's of pointers and addresses (3)

Example 1:

```
1.  int v1=4;
2.  int v2=8;
3.  int * p1=&v1;
4.  int v3=6;
5.  int * p2=&v2;


6.  cout << (long)&v1  << " " << (long)&v2 << " " << (long)&v3 << endl;  //140724418276108
    140724418276112 140724418276116 //+4 bytes each one (each int is 32 bits)
7.  cout << (long)&p1  << " " << (long)p1 <<  endl;   //140724418276120 (+4 bytes wrt &v3) 140724418276108
8.  cout << (long)&p2  << " " << (long)p2 <<  endl;   //140724418276128 (+8 bytes wrt &p1 since it is a pointer
    address) 140724418276112
9.
10.
11.
```

Arithmetic's of pointers and addresses (4)

Example 2:

```
1.  int v1=4;
2.  int v2=8;
3.  int * p1=&v1;
4.  int v3=6;
5.  int * p2=&v2;


6.  cout << (long)&v1  << " " << (long)&v2 << " " << (long)&v3 << endl;  //140724418276108 140724418276112 140724418276116 //+4 bytes each one (each int is 32 bits)
7.  cout << (long)&p1  << " " << (long)p1 <<  endl;   //140724418276120 (+4 bytes wrt &v3) 140724418276108
8.  cout << (long)&p2  << " " << (long)p2 <<  endl;   //140724418276128 (+8 bytes wrt &p1 since it is a pointer address) 140724418276112
9.
10. cout << (long)p1+1 << endl;      //140724418276109 (address of v1 +1 byte, i.e., it is like "((long)p1)+1" )
11. cout << (long)(p1+1)  << endl;   //140724418276112 (address of v2, i.e., the variable after v1 in the memory) (address of the variable pointed by p1 (v1) + the number of bytes of the type of p1 (4 bytes))
```

Arithmetic's of pointers and addresses (5)

Example 2:

```
1.  char v1=4;
2.  char v2=8;
3.  char * p1=&v1;
4.  char v3=6;
5.  char * p2=&v2;


6.  cout << (long)&v1  << " " << (long)&v2 << " " << (long)&v3 << endl; //140730076670309 140730076670310 140730076670311 //+1 byte (each char is 1 byte)
7.  cout << (long)&p1  << " " << (long)p1 <<  endl;   //140730076670312 (+1 byte wrt &v3) 140730076670309
8.  cout << (long)&p2  << " " << (long)p2 <<  endl;   //140730076670320 (+8 bytes wrt &p1 since it is a pointer address) 140730076670310
9.
10. cout << (long)p1+1 << endl;       //140730076670310 (address of v1 +1 byte)
11. cout << (long)(p1+1)  << endl;    //140730076670310 (address of v1 +1 byte for the type char)
12.
```

Priority between derefence and arithmetic operators

Note

- Attention to the priority between the derefence operator "*" and the arithmetic operators
  - *pv+1 is equivalent to (*pv)+1, while it is not equivalent to *(pv+1)
  - *pv++ is equivalent to *(pv++), while it is not equivalent to (*pv)++

→ It is suggested to use the parenthesis to avoid confusion

Example above:

{ ../RIF_POINT/priority.cc}

{ ../RIF_POINT/priority1.cc}

Arithmetic's of pointers and addresses (1): example

Arithmetic of pointers

Example

```
1.  int x=0;
2.  int *px = &x;

3.  cout << "Address(x) = " << (long) &x << endl;       //address of x, e.g.,  140723941814244
4.  cout << "Size(int) = " << sizeof(int) << endl;      //4
5.  cout << "Address(px) = " << (long) px << endl;      //address of x, e.g., 140723941814244
6.  cout << "Address(&px) = " << (long) &px << endl;    //address of px, e.g., 140723941814248
7.
8.
9.
```

Arithmetic's of pointers and addresses (1): example

Arithmetic of pointers

Example

```
1.  int x=0;
2.  int *px = &x;

3.  cout << "Address(x) = " << (long) &x << endl;       //address of x, e.g.,  140723941814244
4.  cout << "Size(int) = " << sizeof(int) << endl;      //4
5.  cout << "Address(px) = " << (long) px << endl;      //address of x, e.g., 140723941814244
6.  cout << "Address(&px) = " << (long) &px << endl;    //address of px, e.g., 140723941814248
7.  px += 2;  //address(px) + (2*sizeof(int)) = 140723941814244 + 8 (bytes)
8.  cout << "Address(px) = " << (long) px << endl;      //new address, e.g., 140723941814252
9.
```

```
cout<< *px;  //it can print unexpected values
```